

S2.01 - Développement d'une application

Find My Word

Noms / Kardava Sandro / Kaddour Soulayman
Prénoms :
Groupe : Cérés

1. Comment lancer le programme

Le programme se lance depuis un terminal, sans avoir besoin d'ouvrir un environnement de développement. Il faut se placer dans le dossier FindMyWord/ du projet.

Étape 1 — Compilation (place les fichiers .class dans le dossier bin/) :

Linux / macOS :

```
javac -cp lib/wordset.jar -d bin src/game/*.java
```

Étape 2 — Lancement :

Linux / macOS :

```
java -cp "bin:lib/wordset.jar" game.Main
```

Remarque : le chemin data/mots.json est relatif au dossier depuis lequel on lance la commande; il faut donc bien se trouver dans FindMyWord/.

2. Présentation du jeu

Find My Word est une version simplifiée du jeu Wordle. Le programme choisit un mot secret de 5 lettres (sans lettre répétée), et le joueur dispose de 6 tentatives pour le deviner. Après chaque tentative valide, le programme indique pour chaque lettre si elle est bien placée (OK), présente mais mal placée (PRESENT), ou absente du mot (ABSENT). Le joueur marque des points en fonction de la rapidité avec laquelle il trouve le mot.

3. Modélisation UML

La modélisation a été réalisée avant l'implémentation. Conformément à la méthode demandée, nous présentons d'abord un diagramme simplifié (uniquement les liens entre classes), puis un diagramme détaillé (avec les attributs et les opérations).

3.1 Diagramme de classes détaillé

Le diagramme détaillé reprend la même structure en ajoutant, pour chaque classe, ses attributs et ses opérations (y compris les constructeurs). L'énumération `ResultatLettre` et l'interface `WordRepository` sont signalées par leur stéréotype.

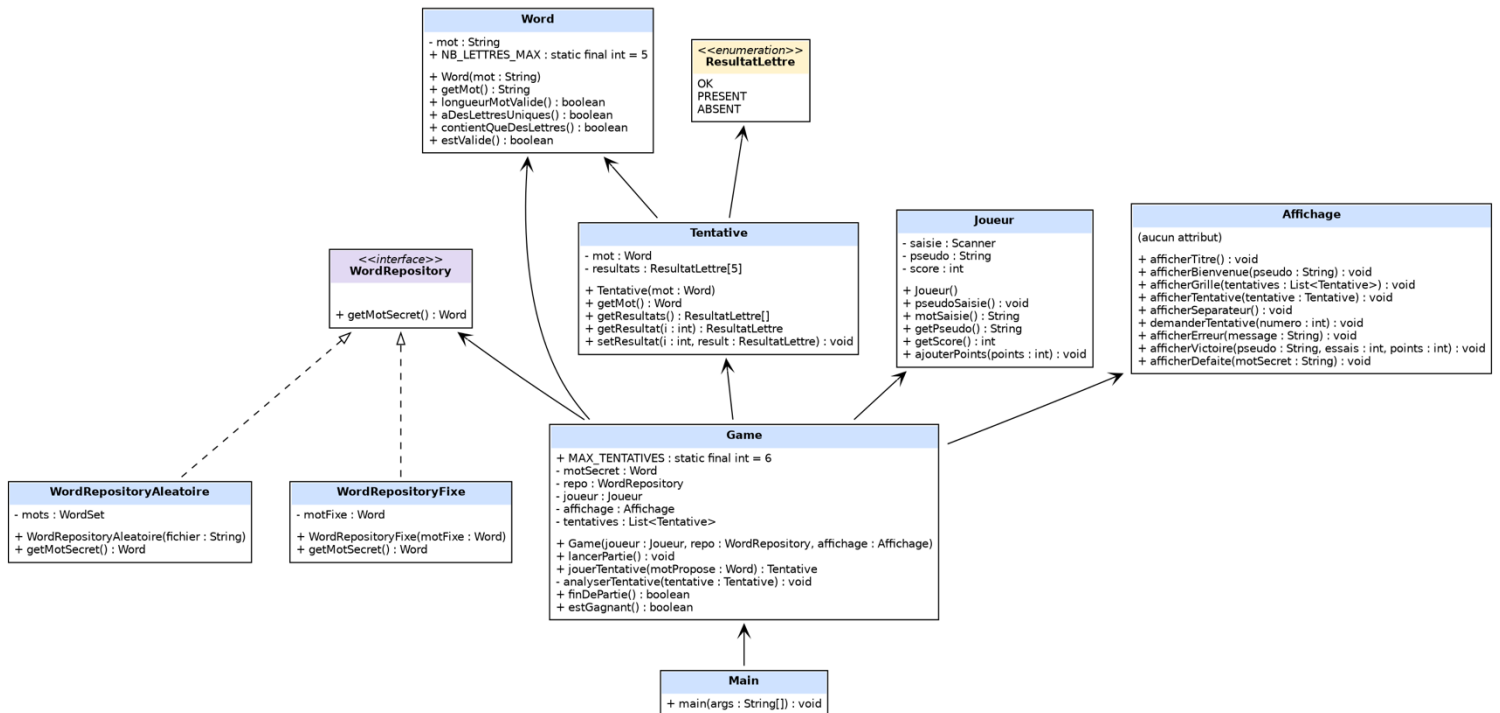


Figure 2 — Diagramme de classes détaillé

4. Choix de conception

4.1 Séparation des responsabilités

Chaque classe a une responsabilité unique et clairement identifiée :

Word : représente un mot et encapsule les règles de validité (longueur, lettres uniques, caractères alphabétiques)

Tentative : associe un mot proposé au résultat de l'analyse de chacune de ses lettres

ResultatLettre : énumération des trois résultats possibles : OK, PRESENT, ABSENT

Game : gère la logique de la partie : mot secret, validation, analyse, suivi des essais, fin de partie

Joueur : gère le pseudo, le score et la saisie au clavier

Affichage : regroupe tout l'affichage texte (grille, messages, erreurs)

WordRepository : interface décrivant une source de mot secret

WordRepositoryAleatoire / WordRepositoryFixe : deux implémentations : tirage aléatoire (jeu réel) et mot fixe (tests)

Main : point d'entrée qui assemble les objets et lance la partie

4.2 Encapsulation

Tous les attributs sont déclarés private. L'accès se fait uniquement par des méthodes (getters et méthodes métier). Par exemple, la validité d'un mot n'est pas calculée à l'extérieur de la classe Word mais par sa méthode estValide()

4.3 Héritage et polymorphisme

L'interface WordRepository est implémentée par deux classes. La classe Game manipule un WordRepository sans savoir s'il s'agit d'une source aléatoire ou fixe : c'est le polymorphisme. Cela permet de tester facilement le jeu avec un mot connu, sans modifier Game.

4.4 Séparation logique / affichage

La logique du jeu (classe Game) est séparée de la présentation (classe Affichage). Game ne fait aucun System.out.println de présentation directement : il délègue à Affichage. Cela respecte la consigne du sujet.

4.5 Analyse des lettres

Comme le sujet précise qu'une lettre apparaît au maximum une fois dans le mot, l'analyse est simple : pour chaque position, si la lettre est identique à celle du mot secret, le résultat est OK ; sinon, si la lettre existe ailleurs dans le mot secret, le résultat est PRESENT ; sinon ABSENT.

4.6 Utilisation de la bibliothèque fournie

La lecture du fichier JSON de mots est confiée à la bibliothèque fournie (wordset.jar). La classe WordRepositoryAleatoire utilise JsonWordSet et sa méthode random() pour obtenir un mot secret aléatoire.

5. Bilan du projet

5.1 Ce qui a été fait et qui fonctionne

- Choix aléatoire d'un mot secret de 5 lettres dans le fichier JSON
- Saisie du pseudo du joueur
- Affichage de la grille et des tentatives déjà jouées
- Validation des tentatives (longueur, caractères, lettres répétées) avec messages d'erreur
- Une saisie invalide n'est pas comptée comme un essai
- Analyse correcte de chaque lettre (OK / PRESENT / ABSENT)
- Gestion du nombre d'essais (maximum 6)
- Détection de la fin de partie (victoire ou défaite)
- Calcul du score (6 points au 1er essai, ... 1 point au 6e)
- Affichage d'un message de fin clair, avec le mot secret révélé en cas de défaite

5.2 Ce qui n'a pas été fait

- Les fonctionnalités optionnelles n'ont pas été implémentées : gestion du temps, enchaînement de plusieurs tours entre deux joueurs, et historique des parties.
- Le score est calculé et stocké dans le joueur, mais comme une seule partie est jouée, il n'y a pas de classement entre plusieurs manches.

5.3 Tests réalisés

Les cas de test de l'annexe B ont servi de référence. La classe `WordRepositoryFixe` permet de fixer le mot secret (par exemple « ligne ») pour vérifier que l'analyse correspond bien aux sorties attendues du sujet.

5.4 Organisation du travail

Au tout début, nous avons commencé par lire le sujet ensemble et réaliser le diagramme UML à deux, car c'était la partie la plus importante pour bien se mettre d'accord sur les classes avant de coder. Une fois la modélisation validée, nous avons réparti l'écriture des classes en suivant le découpage du diagramme.

La répartition a été la suivante :

Kardava Sandro s'est occupé des classes liées au mot et aux données : `Word`, `Tentative`, `ResultatLettre`, l'interface `WordRepository` et ses deux implémentations (`WordRepositoryAleatoire` et `WordRepositoryFixe`).

Kaddour Soulayman s'est occupé du déroulement de la partie et des interactions : `Game`, `Joueur`, `Affichage` et `Main`.

Comme la classe `Game` utilise presque toutes les autres classes, nous avons souvent dû travailler ensemble pour les faire communiquer correctement (par exemple, le type de retour de `jouerTentative()` ou les méthodes attendues dans `Affichage`). Nous nous sommes répartis les fichiers mais nous nous sommes appelés régulièrement pour vérifier que tout restait cohérent, et nous avons relu le code de l'autre avant de l'assembler. Chacun a donc une bonne compréhension de l'ensemble du projet, même des parties qu'il n'a pas écrites lui-même.

Concernant les choix réalisés : nous avons décidé de garder le strict nécessaire demandé dans le sujet et de ne pas implémenter les fonctionnalités optionnelles (temps, plusieurs manches, historique), afin de rendre un code propre et bien testé plutôt qu'un code plus complet mais moins soigné. Nous avons aussi choisi d'ajouter une classe `WordRepositoryFixe` dès le début, car elle nous a beaucoup aidés à tester le jeu avec un mot connu (« ligne ») sans dépendre du tirage aléatoire.

5.5 Difficultés rencontrées

La première difficulté a été d'utiliser le fichier `.jar` fourni. Au départ, le programme compilait mais plantait au lancement avec une erreur « `class not found` ». Nous avons compris qu'il fallait ajouter le `jar` au *classpath* avec l'option `-cp`, à la fois à la compilation et à l'exécution.

La deuxième difficulté a été la logique d'analyse des lettres (OK / PRESENT / ABSENT). Au début nous avons inversé l'ordre des tests, ce qui donnait PRESENT au lieu de OK pour une lettre bien placée. Nous avons compris qu'il fallait d'abord tester si la lettre est à la bonne position (OK), puis seulement après si elle est présente ailleurs (PRESENT). Le fait que le sujet précise qu'une lettre n'apparaît jamais deux fois nous a bien simplifié ce travail.

La troisième difficulté a été de séparer la logique de l'affichage. Au premier essai, nous avons mis beaucoup de `System.out.println` directement dans `Game`. Comme le sujet demande de séparer le calcul de la présentation, nous avons déplacé tous les affichages dans la classe `Affichage`, et `Game` se contente maintenant de lui demander d'afficher. Cela a rendu le code de `Game` beaucoup plus court et plus lisible.

Enfin, une petite difficulté a été de bien faire la différence entre une saisie invalide (qui ne doit pas compter comme un essai) et une tentative valide. Nous avons réglé cela avec une boucle qui redemande la saisie tant que le mot n'est pas valide, sans rien ajouter à la liste des tentatives.